

มหาวิทยาลัยมหิดล
Mahidol University

ห้ามมิให้นักศึกษาทำซ้ำ ดัดแปลง หรือใช้ประโยชน์จากงานอันมีลิขสิทธิ์ปรากฏอยู่ในระบบการศึกษาอิเล็กทรอนิกส์ (E-Learning) ของมหาวิทยาลัย ไม่ว่าจะทั้งหมดหรือบางส่วน โดยไม่ได้รับอนุญาตจากมหาวิทยาลัย นอกเหนือจากการศึกษาส่วนบุคคล ทั้งนี้ การทำซ้ำ ดัดแปลง หรือเผยแพร่ต่อสาธารณชนซึ่งงานอันมีลิขสิทธิ์ **จะมีโทษปรับตั้งแต่ 20,000 บาท ถึง 200,000 บาท และหากเป็นการกระทำเพื่อการค้า จะมีโทษจำคุกตั้งแต่ 6 เดือน ถึง 4 ปี หรือปรับตั้งแต่ 100,000 บาท ถึง 800,000 บาท หรือทั้งจำทั้งปรับ**

Unless allowed by Mahidol University, the User shall not copy, modify, or exploit in part or in whole of the copyrighted materials on the Platform, other than for your own individual study. The copy, modify or communication to public of the copyrighted materials shall be inflicted with a fine from 20,000 Baht up to 200,000 Baht. If the offence is committed with the commercial purpose, the offender shall be inflicted with imprisonment for a term from 6 months up to 4 years or a fine from 100,000 Baht up to 800,000 Baht or both imprisonment and fine.

Mahidol University
Faculty of Information and Communication Technology



ITDS261 Introduction to Software Engineering

Lecture 9 – Design Architecture and Methodology

Topics

- Architectural vs. detailed design
- Basic techniques for detailed design
- Principles of software engineering design
- Design characteristics

Types of Bridges

BEAM BRIDGE



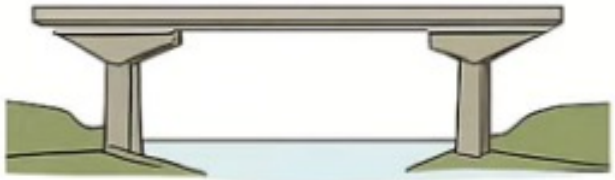
ARCH BRIDGE



TRUSS BRIDGE



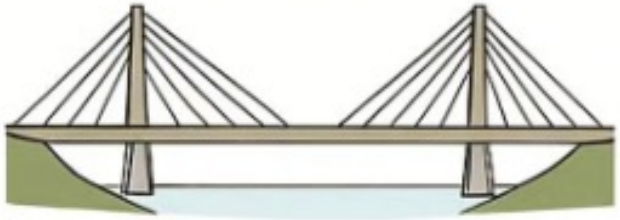
CANTILEVER BRIDGE



SUSPENSION BRIDGE



CABLE-STAYED BRIDGE



And several more types of bridges…

Reference: Engineering Infinity Facebook Page

How to write HelloWorld

1

```java
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello World!");
    }
}
```

2

```java
public class HelloWorld {
    public static void main(String[] args) {
        MessagePrinter printer = new MessagePrinter();
        String message = printer.getMessage();
        System.out.println(message);
    }
}

class MessagePrinter {
    public String getMessage() {
        return "Hello World!";
    }
}
```

3

```java
public class HelloWorld {
    public static void main(String[] args) {
        MessagePrinter printer = new MessagePrinter();
        printer.printMessage();
    }
}

class HelloProvider {
    public String getHello() {
        return "Hello";
    }
}

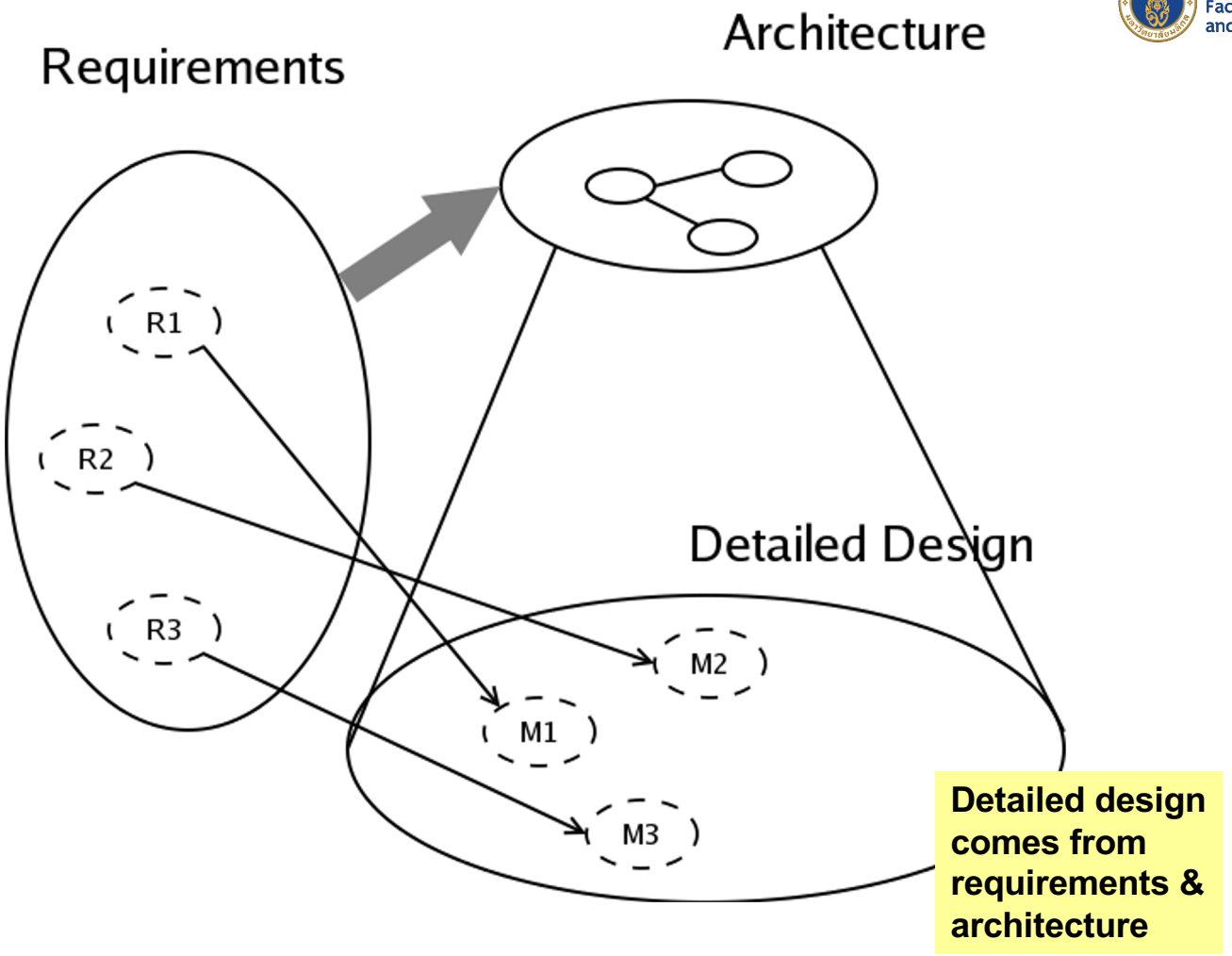
class WorldProvider {
    public String getWorld() {
        return "World";
    }
}

class MessagePrinter {
    private HelloProvider helloProvider = new HelloProvider();
    private WorldProvider worldProvider = new WorldProvider();

    public void printMessage() {
        String message = helloProvider.getHello() + " "
            + worldProvider.getWorld() + "!";
        System.out.println(message);
    }
}
```

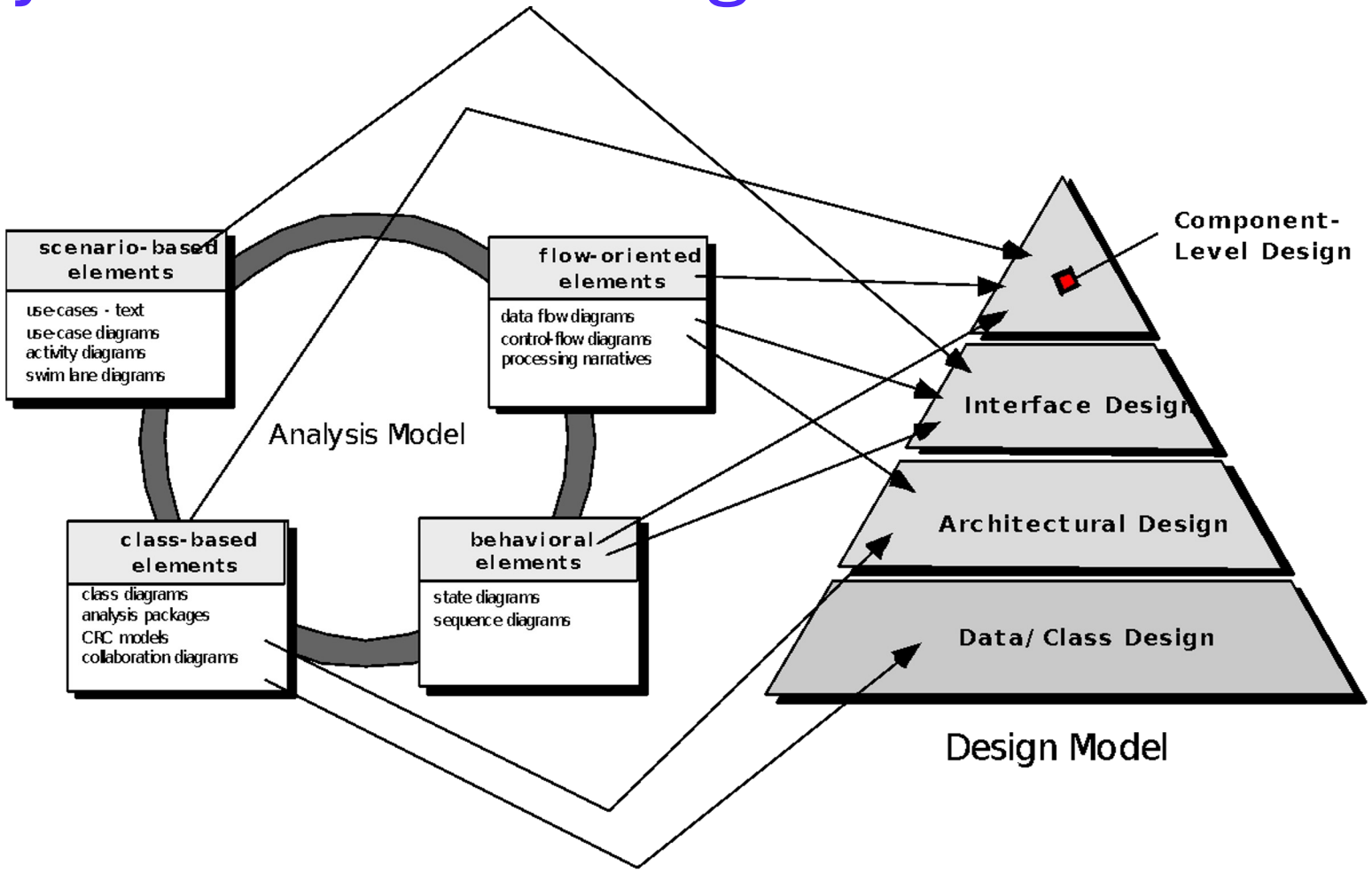
Design Architecture and Methodology

- Start from requirements
- How is the software going to be structured?
  - What are the components?
  - How are these components related?
- Two parts
  - Architectural (high-level) design
  - Detailed design



Relationship between Architecture and Design

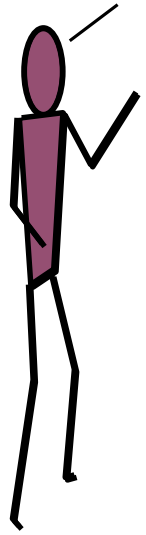
Analysis Model -> Design Model



Source: Roger S. Pressman, "Software Engineering: A Practitioner's Approach", 7th Edition, McGraw Hill, 2009.

An Architectural Design Method

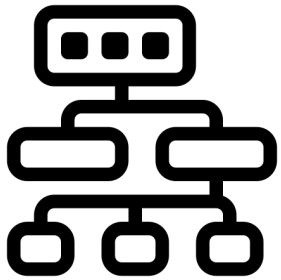
customer requirements



Implementation

Architectural Design

It is an activity of specifying a program's major parts; their responsibilities, properties, and interfaces; and the relationships and interactions among them.



Key Components of Architectural Design:

- Layers & Modules
- Components & Services
- Data Flow & Storage
- Security & Performance
- Scalability & Maintainability
- Integration & APIs



Detailed Design

The detailed design goes in-depth into components, constituting internal elements, and specifies the relationship between them.
The data structures and algorithm specifications are also worked on during detailed design.

Key Components of Detailed Design:

- Module-Level Design
- Class & Object Design
- Function & Algorithm Design
- Database Schema
- Interface & API Design
- Error Handling & Logging
- Security Considerations



Architectural or Detailed Design?

1) The find() operation must use a binary search algorithm.

2) The program must separate the user interface code from the rest of the code.

3) The client must complete processing in no more than one second and the server must complete processing in no more than half a second.

4) The Register class must maintain clients in a hash table.

Architectural or Detailed Design? (Cont.)

5) All utilities in the graphics library must be thread-safe.

6) All Car object must register themselves with the Clock object, and the Clock object will notify the Car objects when a second has passed.

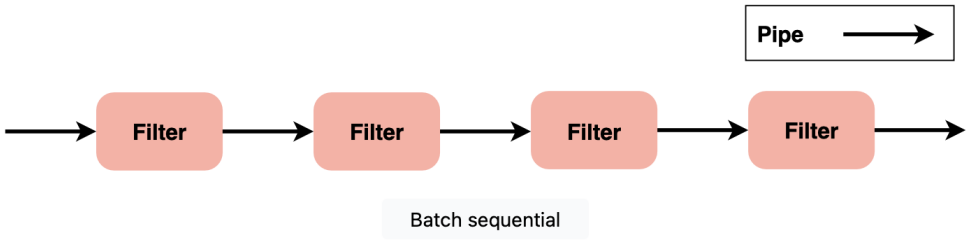
7) All observers of the Clock class must implement the ClockUser class.

Architectural Styles

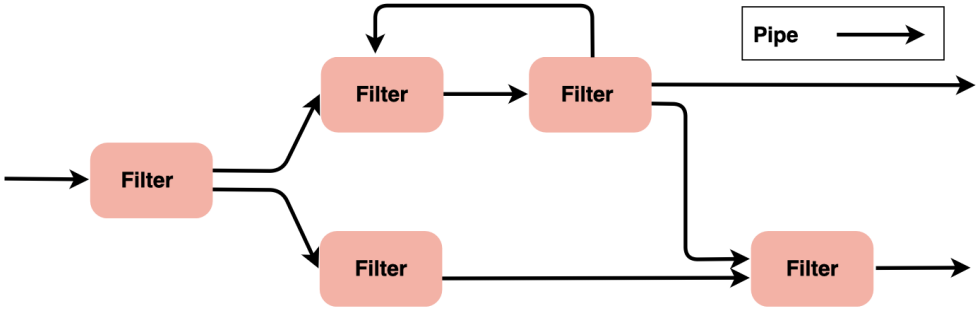
- Batch sequential and Pipe and filter
- Data-centered architecture
- Layered architecture
- Client-Server architecture
- Microservice architecture
- Call and Return architecture

Figures adopted from: https://how.dev/answers/what-are-the-pros-and-cons-of-different-software-architectures

Batch Sequential/Pipe and Filter



Batch sequential

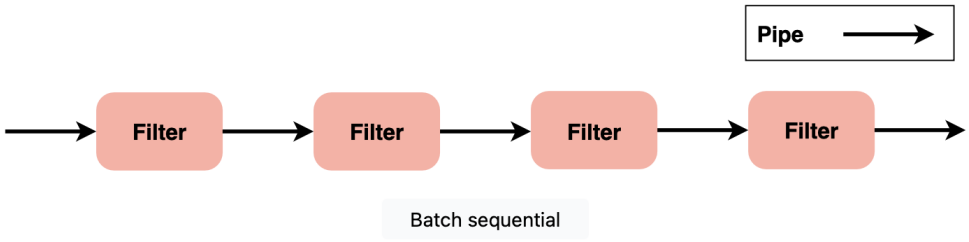


Pipe and filter

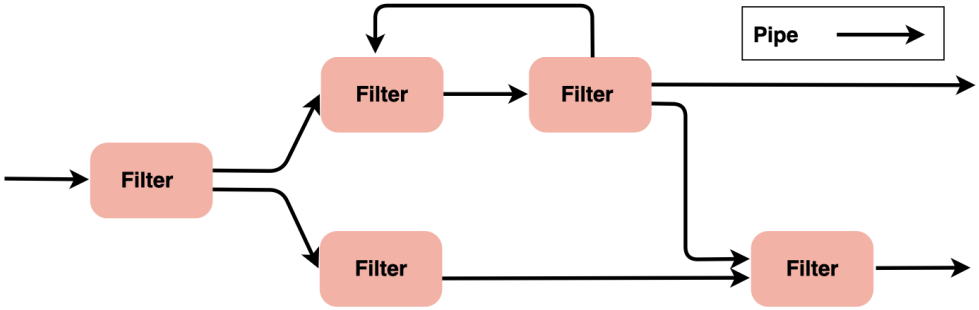
- Data enters the system, and then a series of transformations are applied to the input data before getting the output.
- The system transfers the data with the help of pipes and transforms the data using filters.

Figures adopted from: https://how.dev/answers/what-are-the-pros-and-cons-of-different-software-architectures

Batch Sequential/Pipe and Filter (Cont.)



Batch sequential



Pipe and filter

Pros

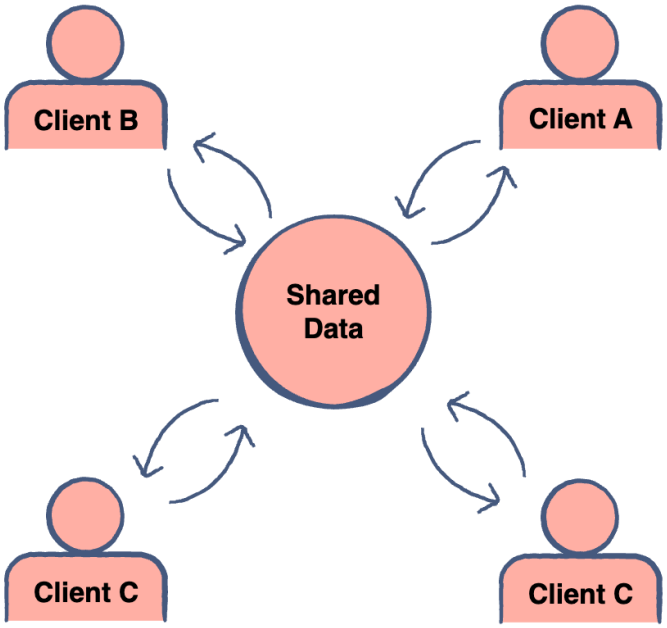
- The same filters can be reused to create different configurations, which allows for reusability.
- Suitable for applications that require a defined series of independent computations to be performed on data.

Cons

- It is not suitable for interactive applications since the output is obtained after a series of transformations.
- Extra filters may be needed to make independent components compatible with one another
- If components are not designed independently, it may lead to tight coupling.

Figures adopted from: https://how.dev/answers/what-are-the-pros-and-cons-of-different-software-architectures

Data-centered Architecture



Data-centered architecture

Shared data is held in a central repository and can be accessed, modified, deleted, or updated by any subsystem.

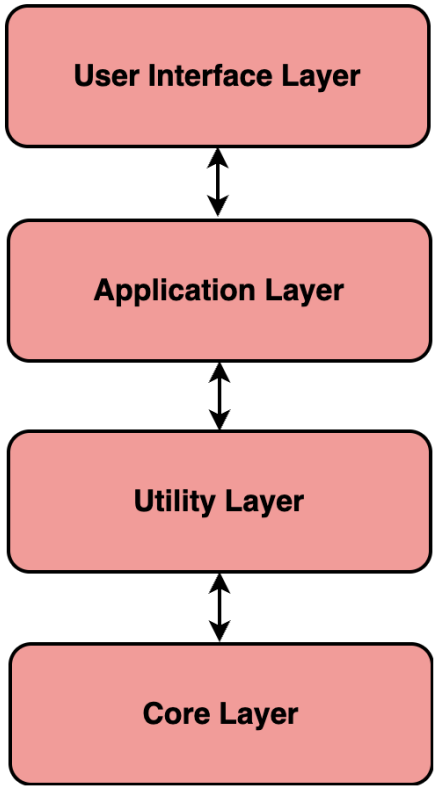
Pros

- Allows efficient sharing of large volumes of data.
- Each component can operate independently.
- Enables consistent data management through centralized backups.

Cons

- Modifying or updating the data model is complex and costly.
- Subsystems must align with a common data model, often requiring compromise.

Layered Architecture



Layered architecture

The system is designed in a stack of layers.
The functionality of a system is organized into layers, with each layer only dependent on the layer below it.

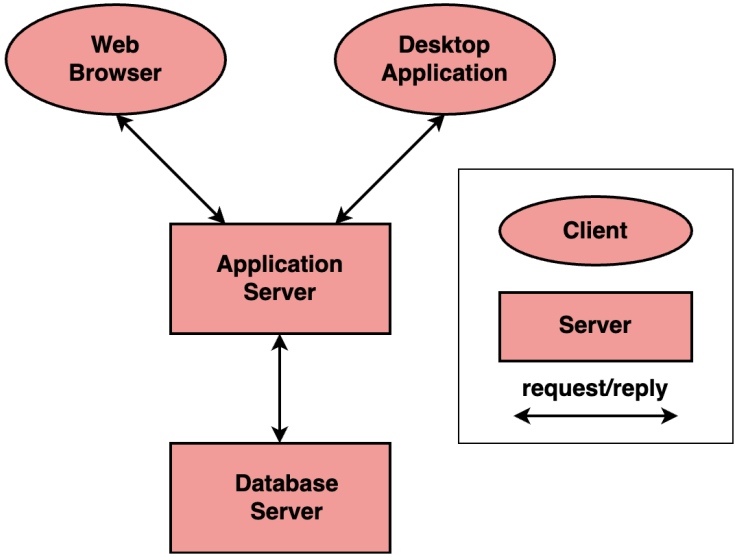
Pros

- Applications built this way are portable across different platforms.
- Helps break down complex problems into manageable parts.
- Layers can be replaced if their interfaces remain unchanged.

Cons

- Achieving a clear separation between layers can be challenging.
- Having many layers can reduce system performance.

Client-Server Architecture



Client-server architecture

The system is divided into the following two components: Client, and Server

The server component is the service provider while clients are the service requesters.

Pros

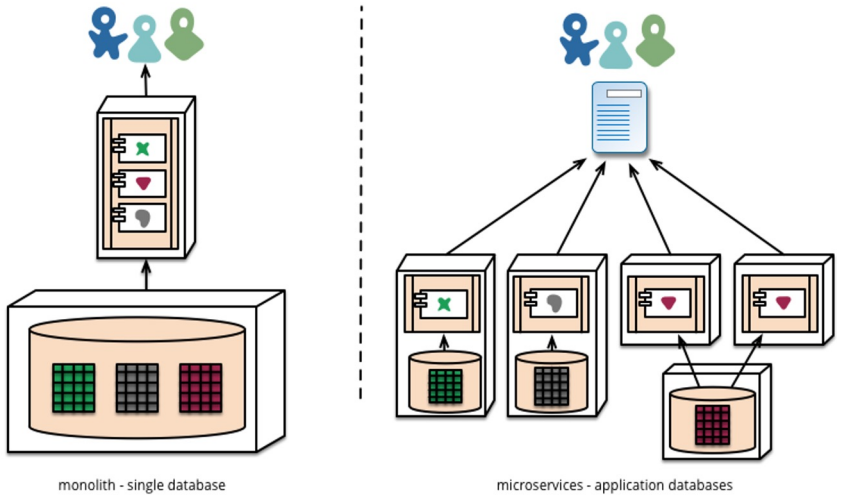
- Allows the use of a distributed system structure.
- Makes it easy to add new servers or upgrade existing ones.

Cons

- Managing the system can be difficult if servers are controlled by third parties.
- Servers can become overwhelmed by too many requests.

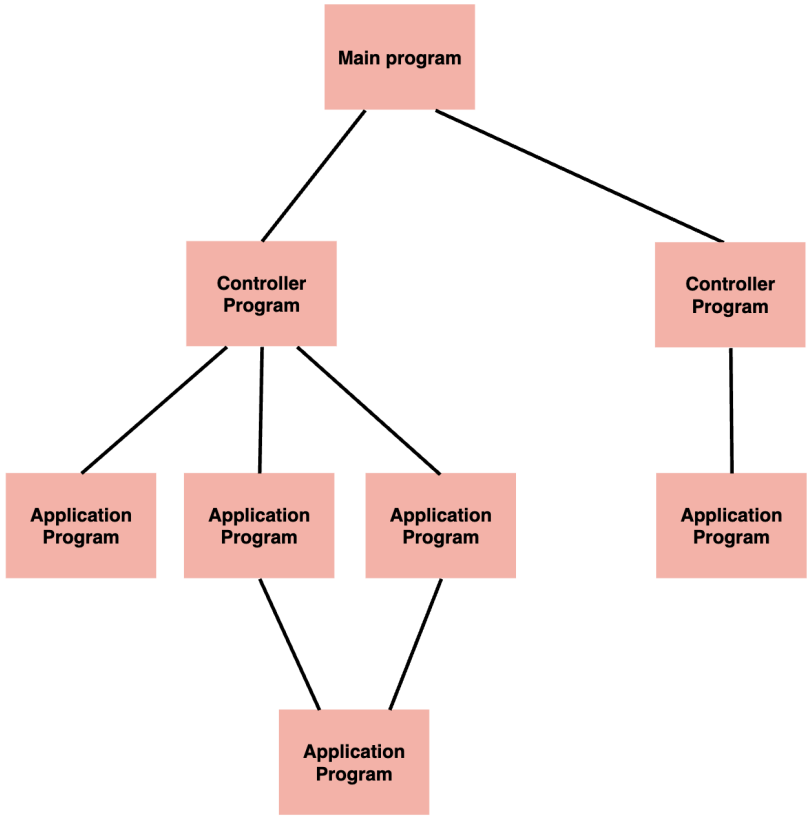
Microservice Architecture

- An approach to develop a single application as a suite of small services.
- Each service:
  - runs in its own process and communicating with lightweight mechanisms, often an HTTP resource API.
  - independently deployable by fully automated deployment machinery.



Source: https://martinfowler.com/articles/microservices.html

Call and Return Architecture

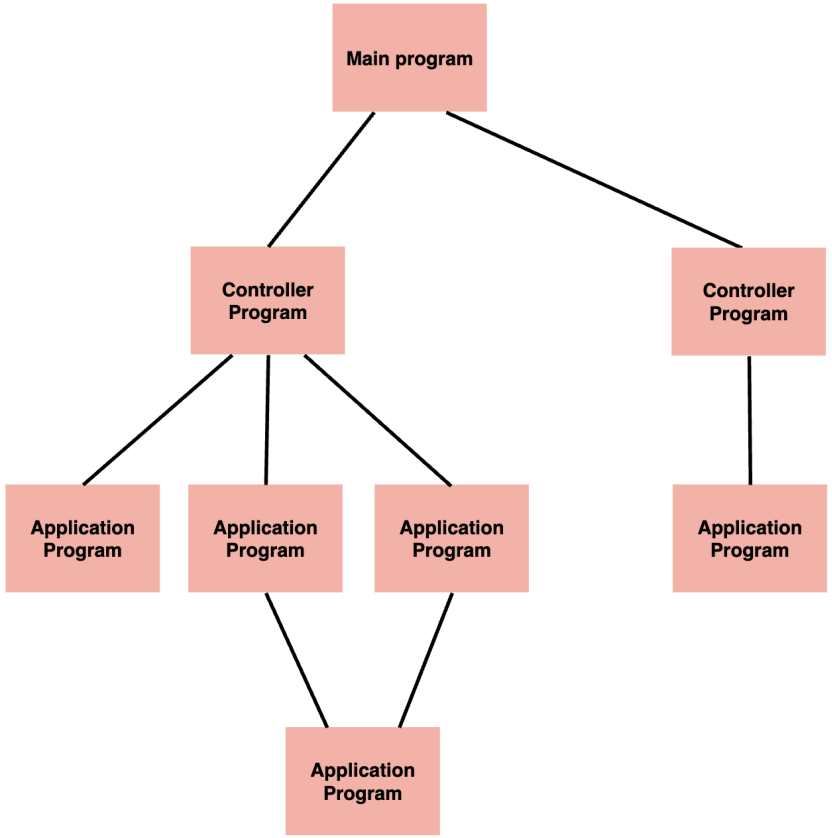


Call and return architecture

Call and return architectures are the most dominant architectures since the inception of software development.

They decompose the main program into a number of program components which, in turn, may invoke other program components.

Call and Return Architecture



Call and return architecture

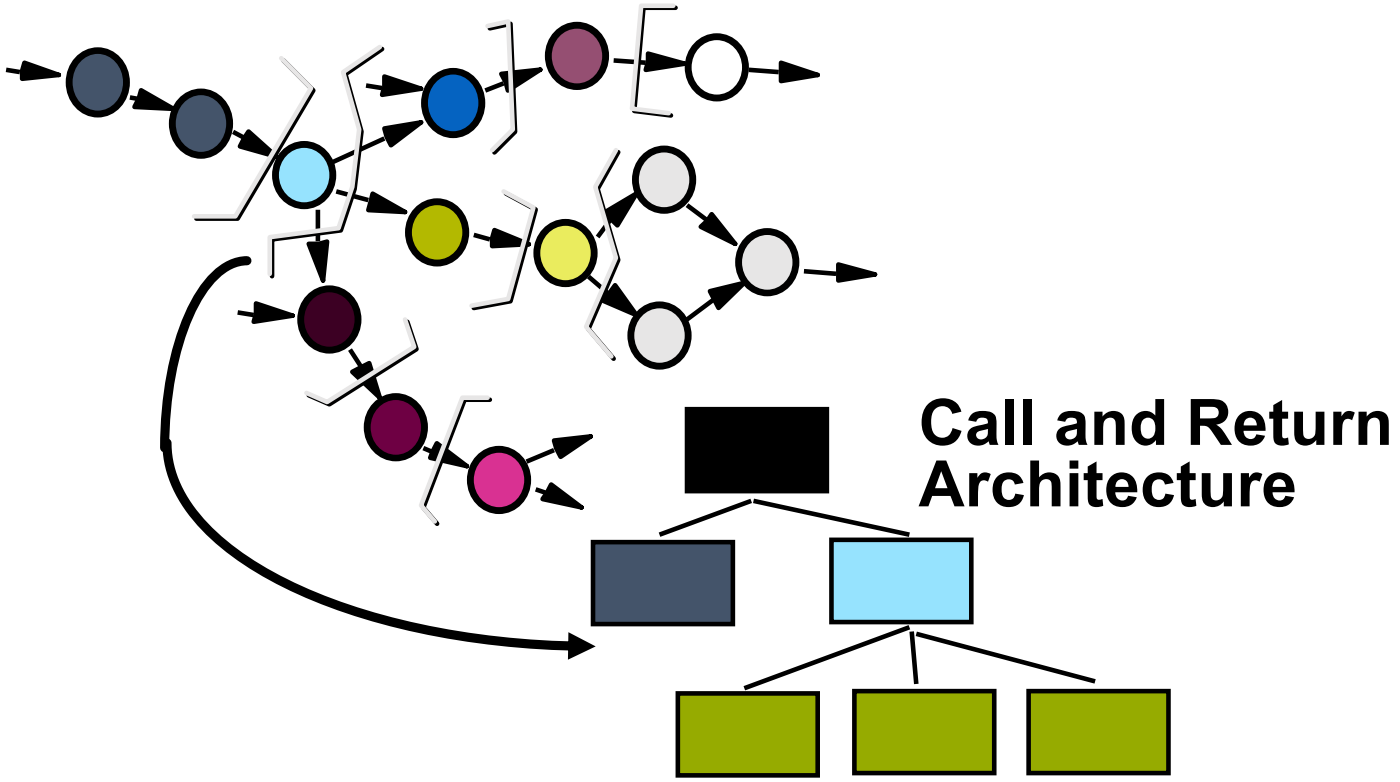
Pros

- Support modular organization of code.
- Easy to scale and make changes when needed.

Cons

- Poorly defined interfaces can cause tight coupling between data.
- Changes to internal data structures may affect other modules.

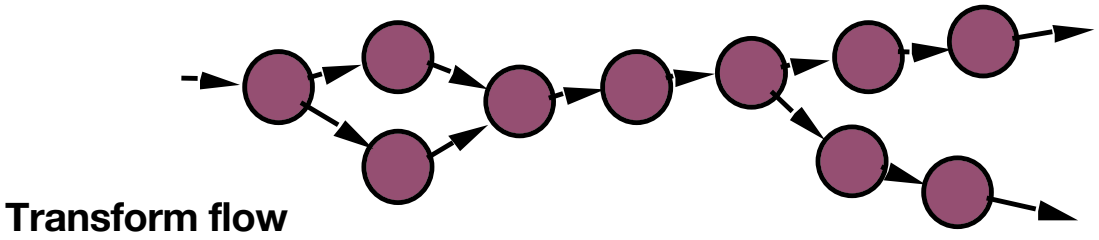
Deriving architectural design



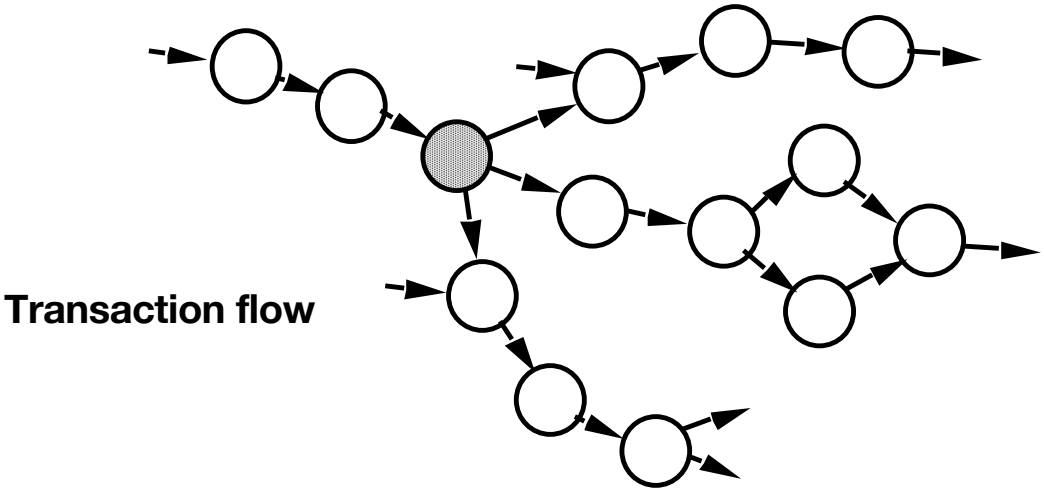
Structured Design

- Objective: to derive a program architecture that is partitioned
- Approach:
  - A DFD is mapped into a program architecture
  - The process specification (PSPEC) and state transition diagram (STD) are used to indicate the content of each module
- Notation: structure chart

Flow Characteristics



Transform flow



Transaction flow

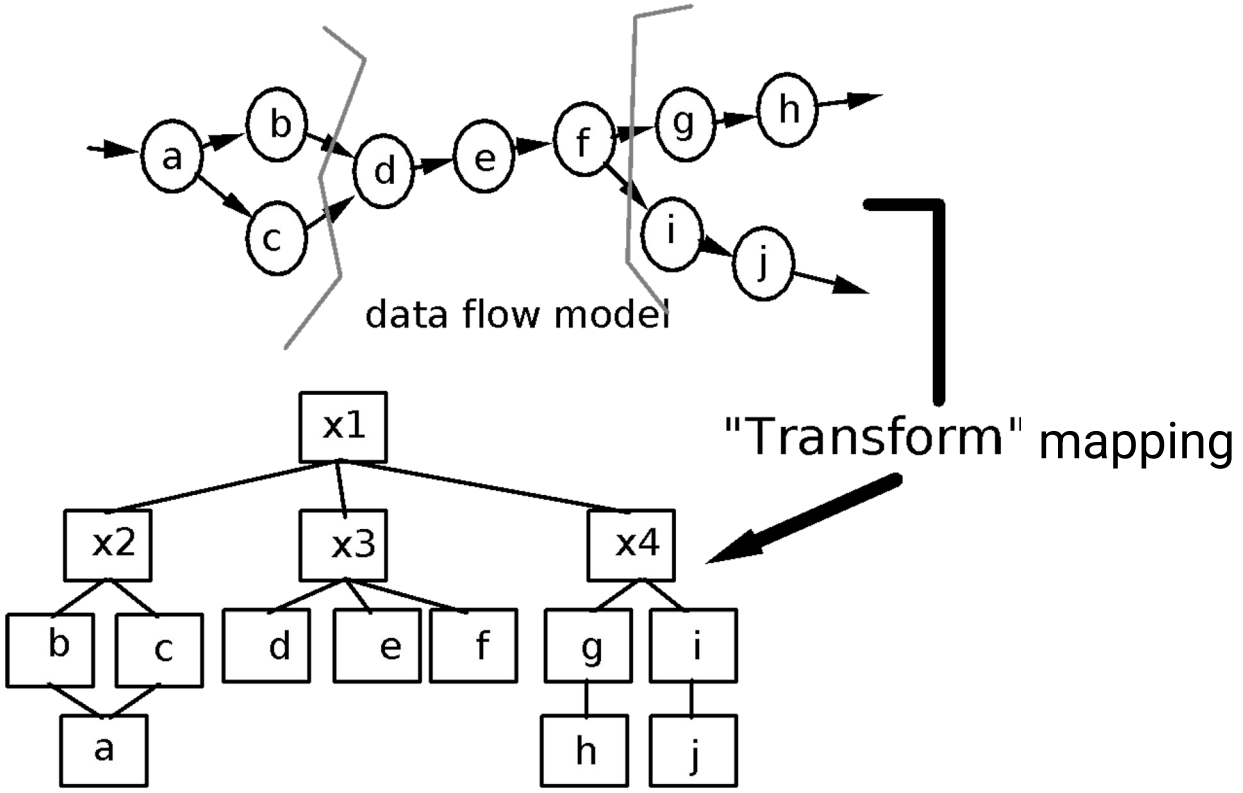
General Mapping Approach

- Isolate incoming and outgoing flow boundaries; for transaction flows, isolate the transaction center.
- Working from the boundary outward, map DFD transforms into corresponding modules.
- Add control modules as required.
- Refine the resultant program structure using effective modularity concepts.

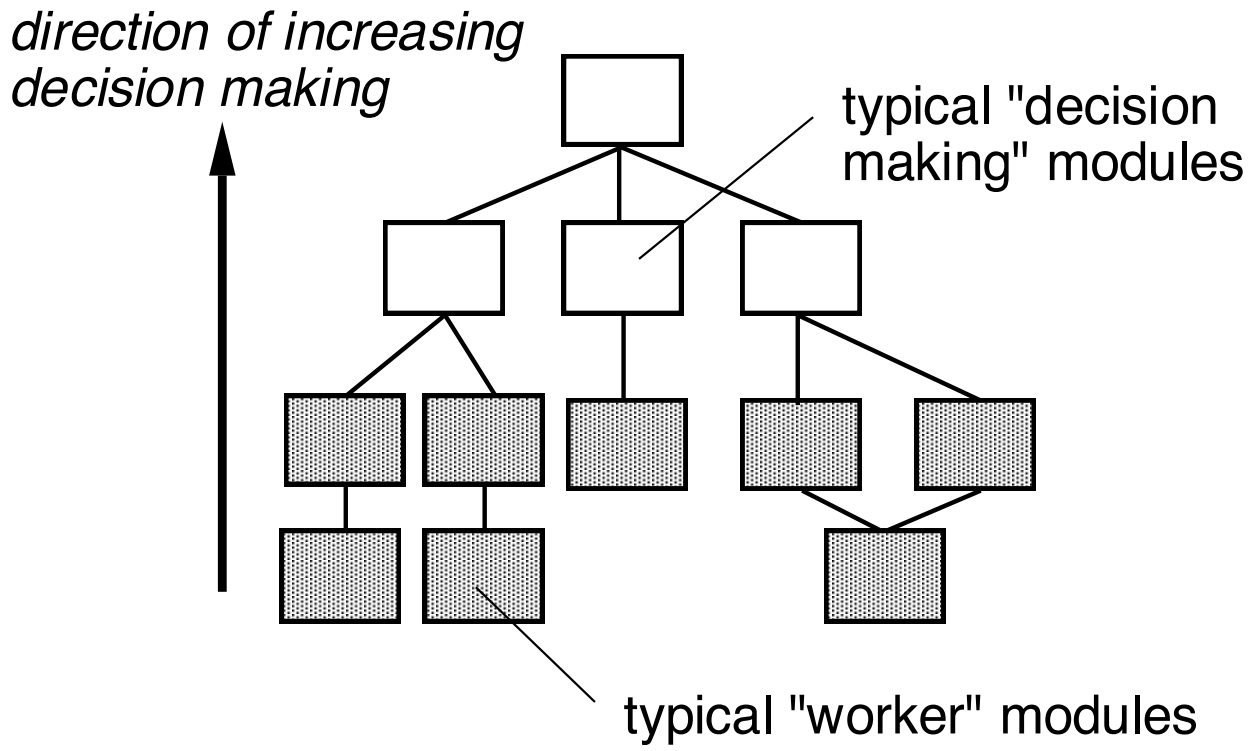
General Mapping Approach (Cont.)

- Perform “first-level factoring”
  - The architecture derived using this mapping results in a top-down distribution of control
  - *Factoring* leads to a program structure in which top-level components perform decision-making and low-level components perform most input, computation, and output work
  - Middle-level components perform some control and do moderate amounts of work
- Perform “second-level factoring”

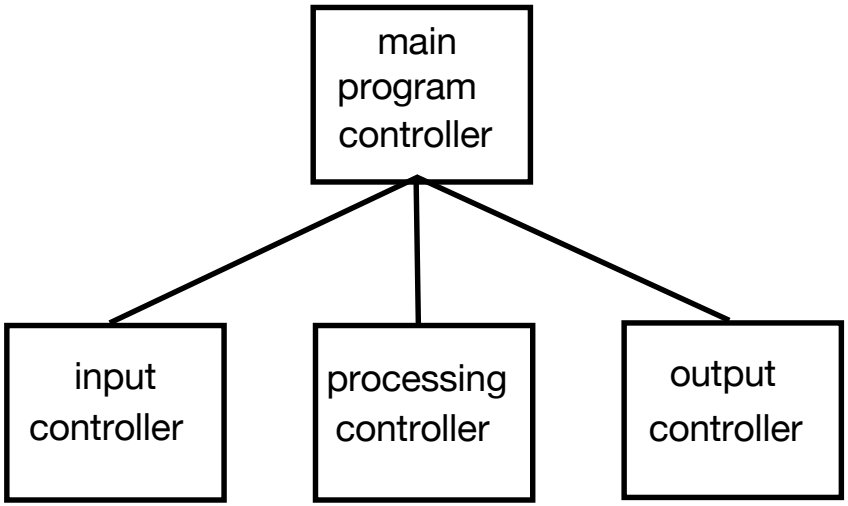
Transform Mapping



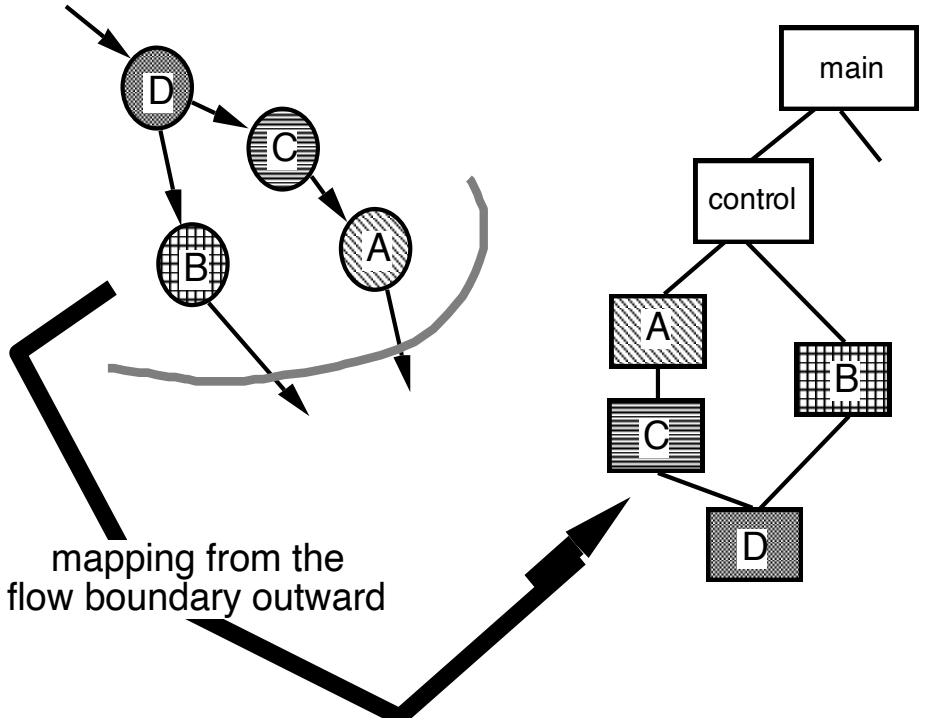
Factoring


direction of increasing
decision making
typical "decision
making" modules
typical "worker" modules

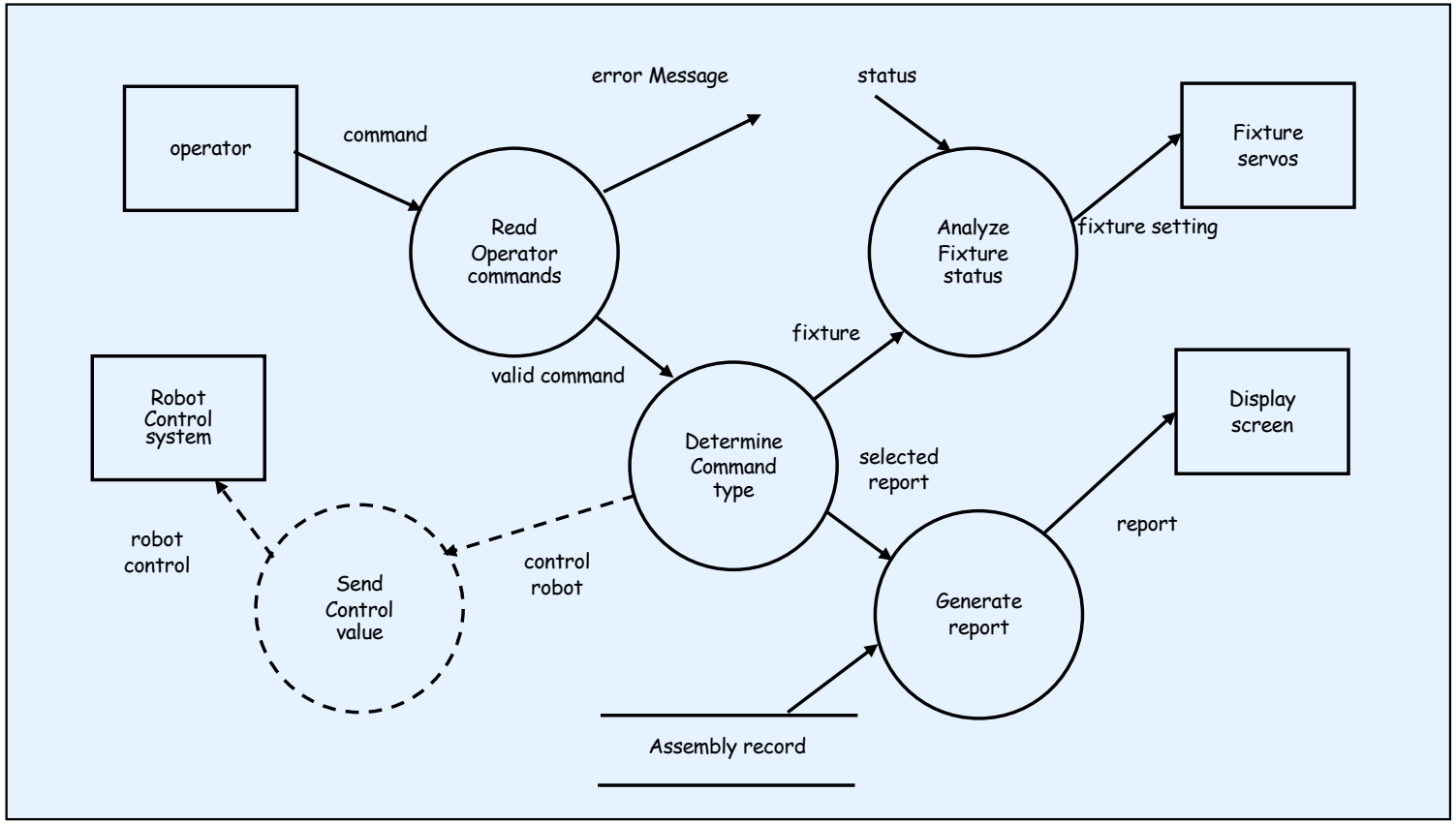
First-Level Factoring



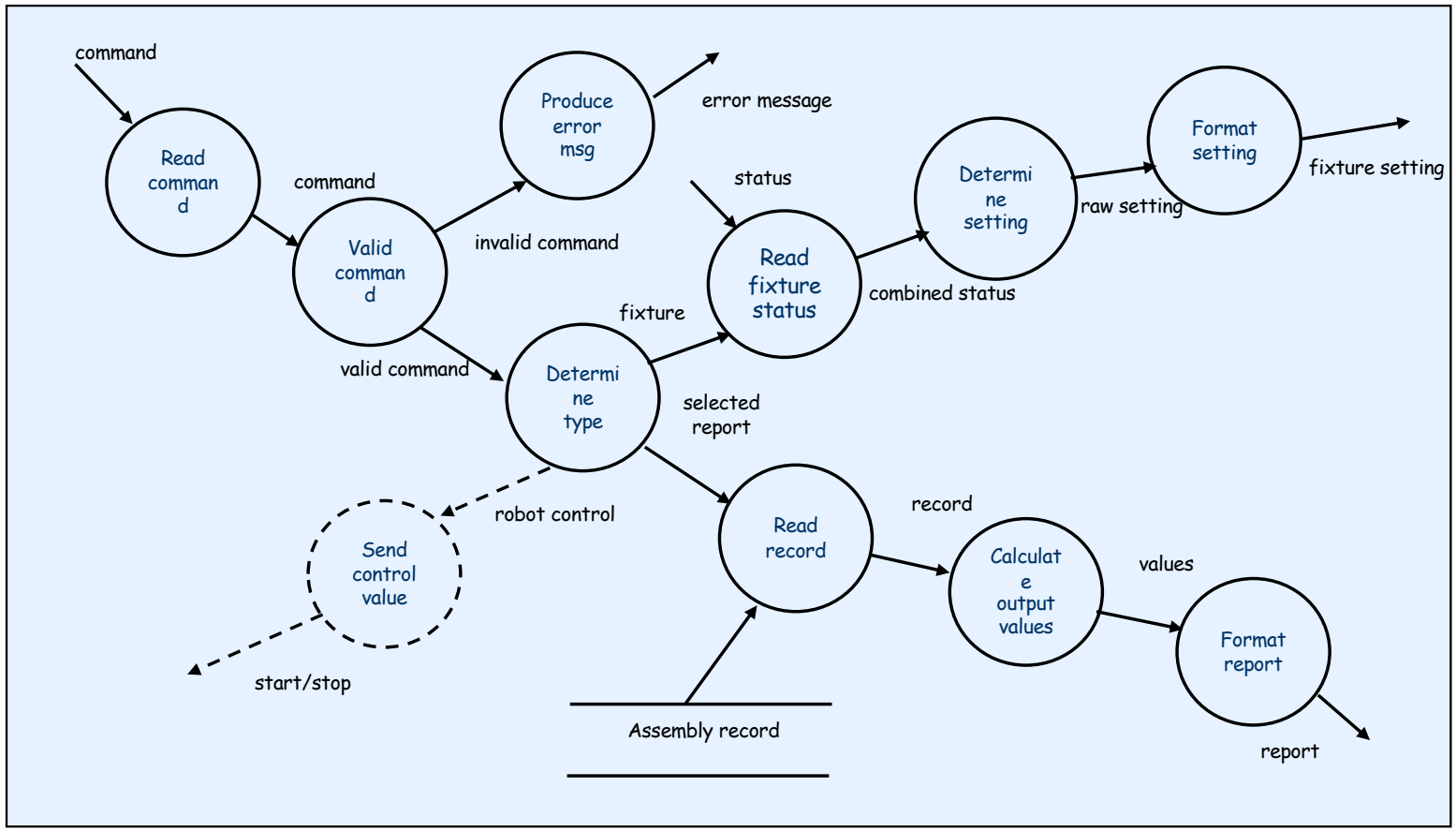
Second-Level Mapping



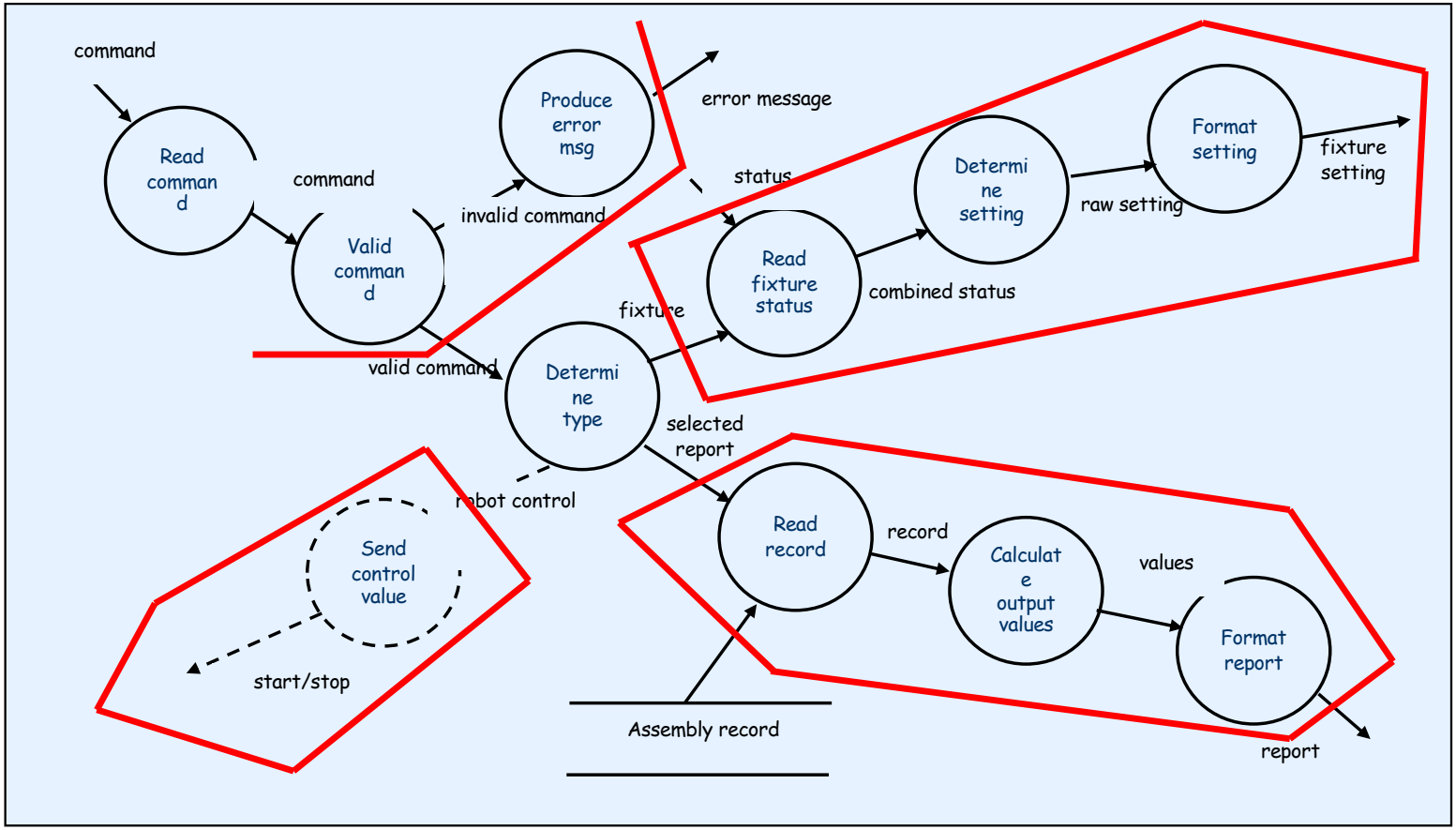
Level 1 Data Flow Diagram



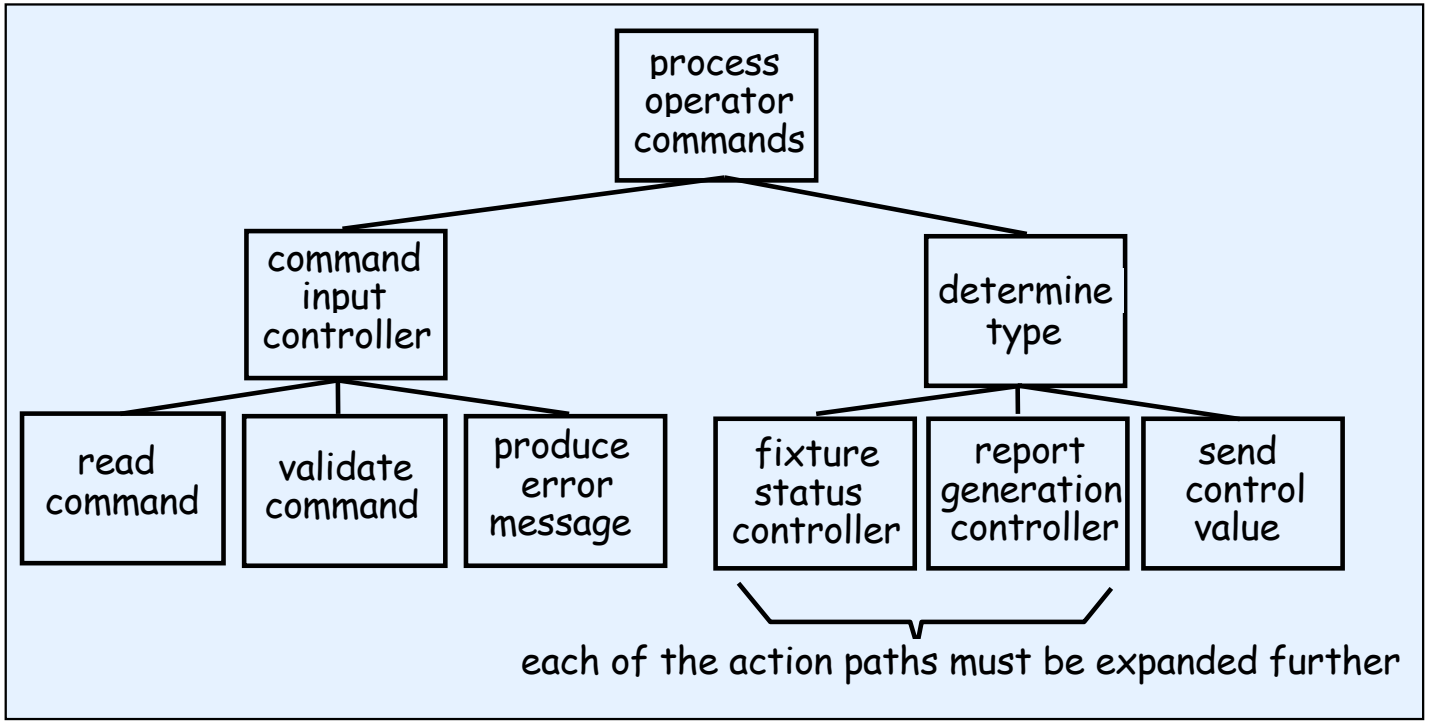
Level 2 Data Flow Diagram



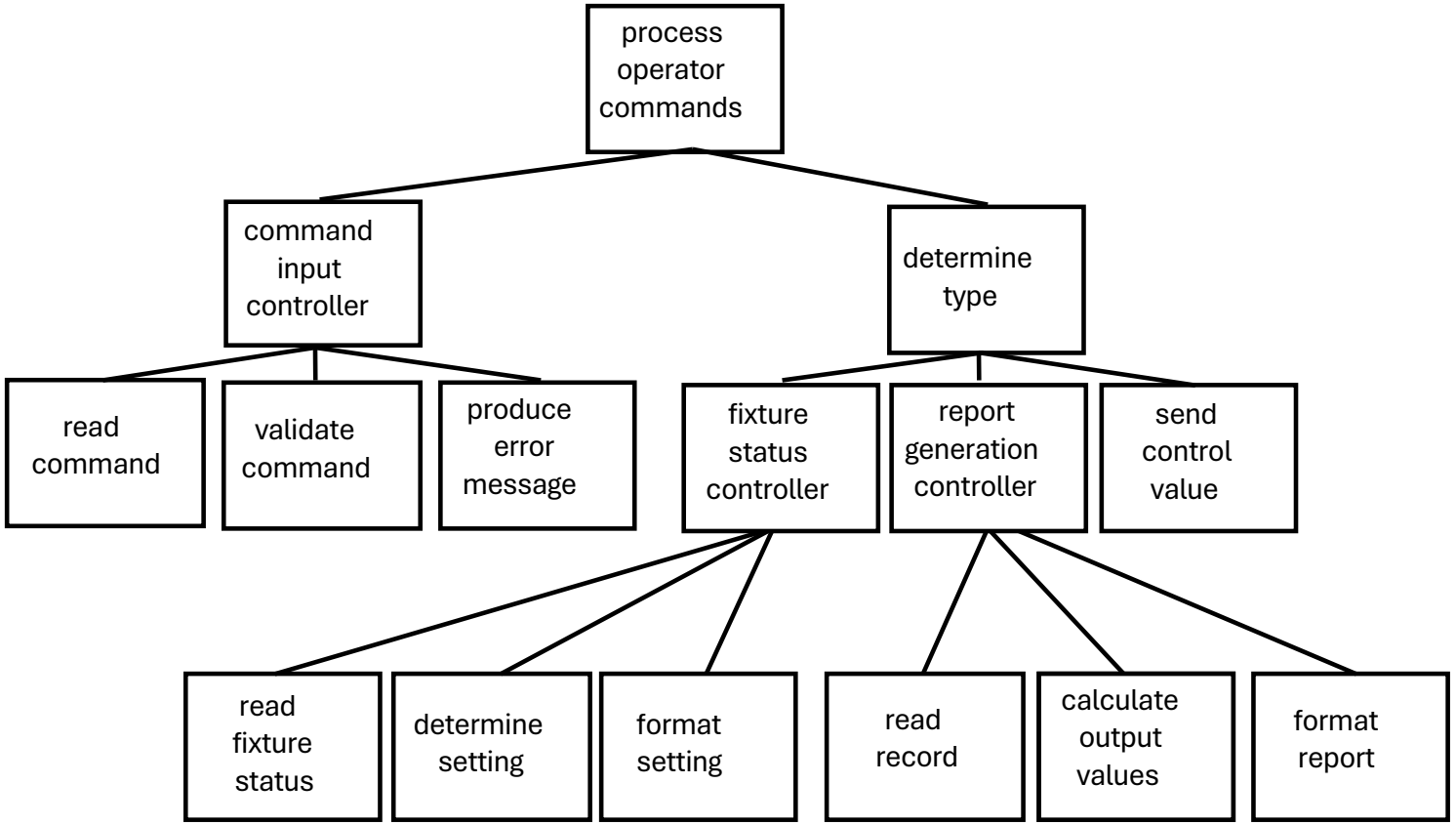
Isolate Flow Paths



Map the Flow Model

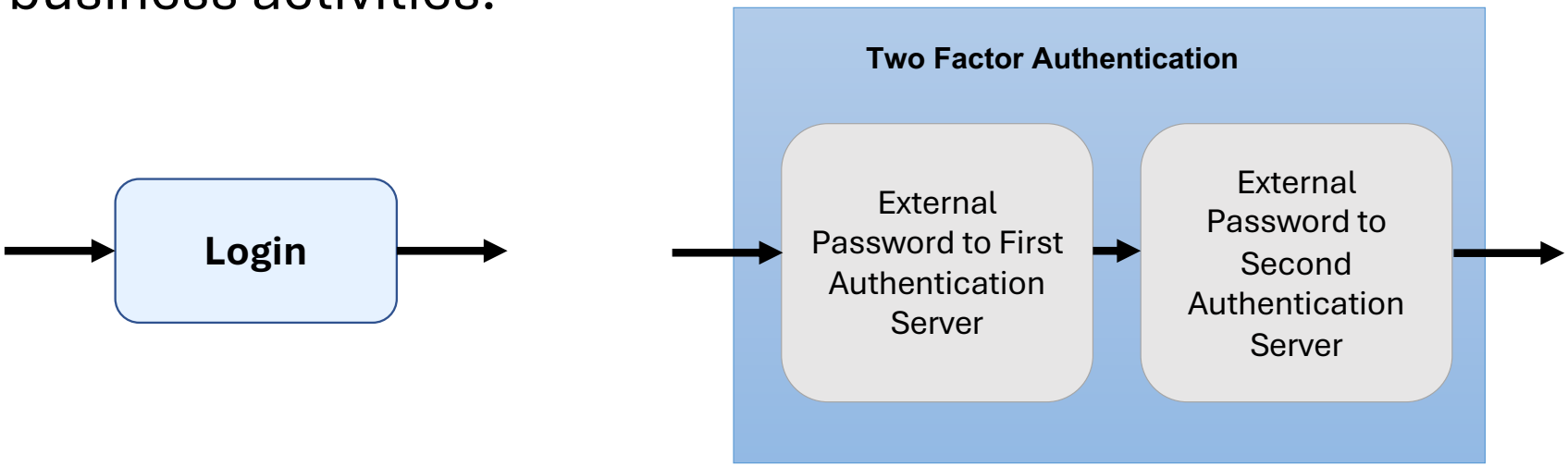


Refining the Program Architecture



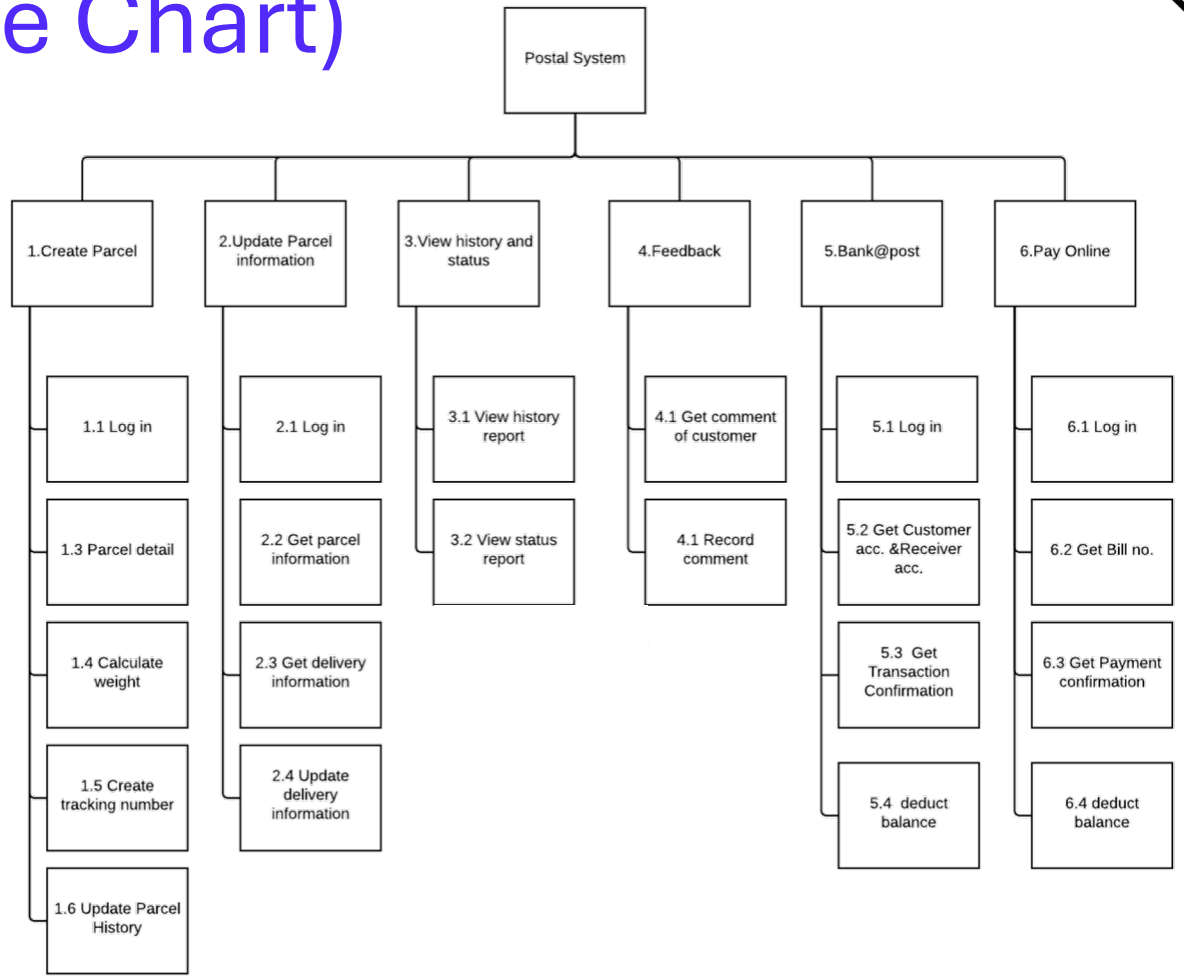
Logical Module vs Physical Module

- A logical module defines the business activities that make up the business processes.
- A physical module defines the physical process to implement business activities.



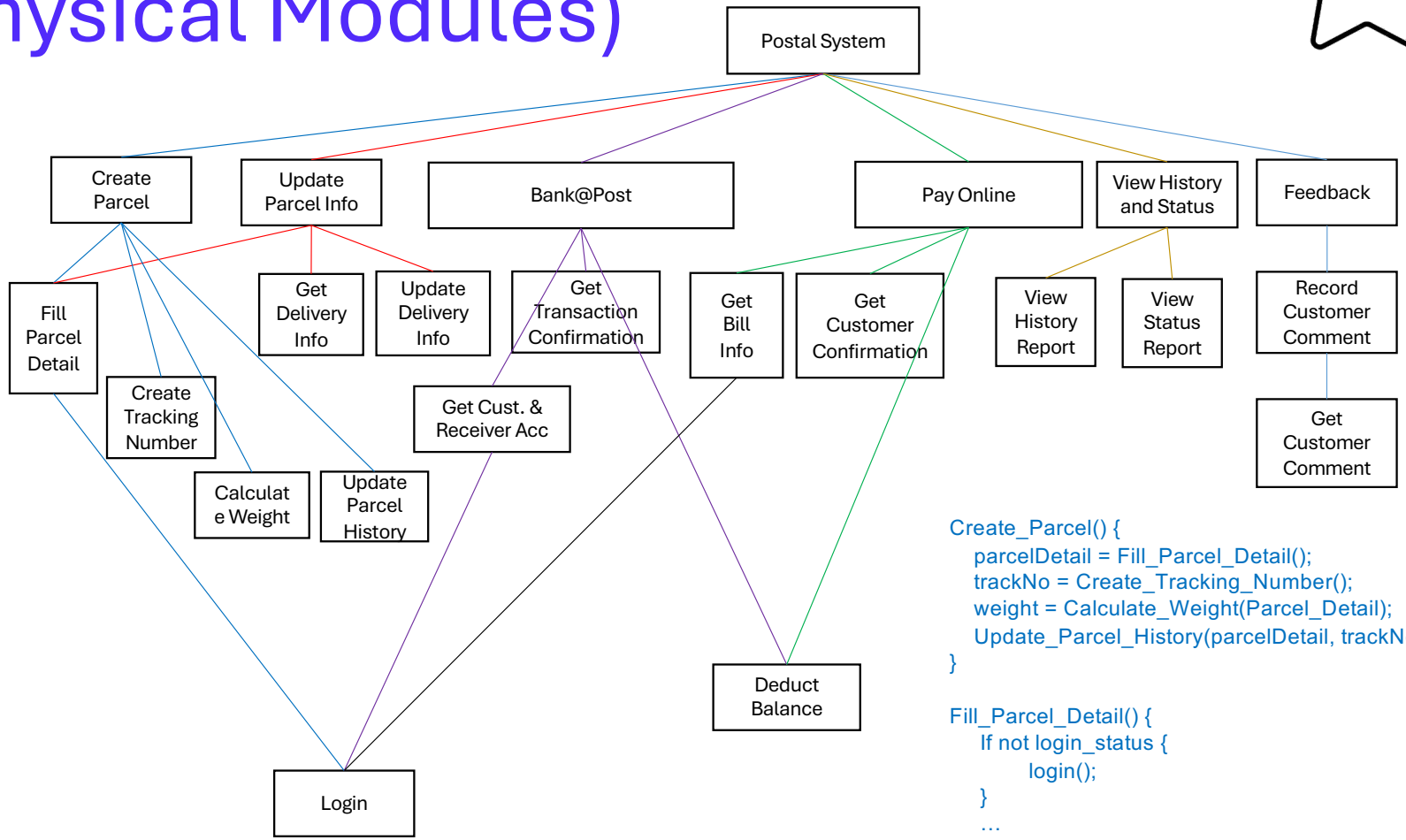
Analysis

Functional Decomposition (Structure Chart)



Structure Chart (Physical Modules)

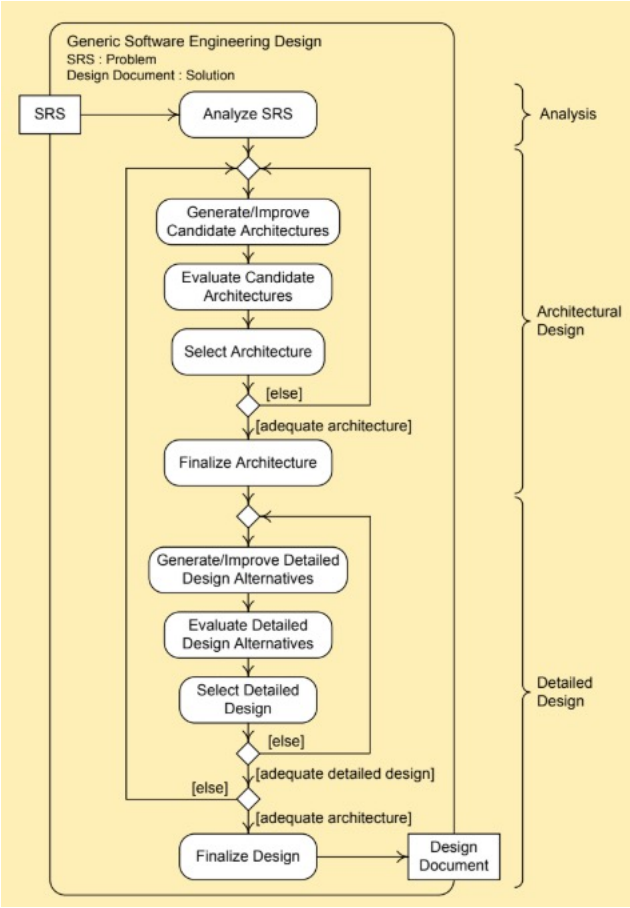
Design




```
Create_Parcel() {
    parcelDetail = Fill_Parcel_Detail();
    trackNo = Create_Tracking_Number();
    weight = Calculate_Weight(Parcel_Detail);
    Update_Parcel_History(parcelDetail, trackNo, weight);
}

Fill_Parcel_Detail() {
     If not login_status {
            login();
     }
     …
     return(parcelDetail);
}
```

Software Engineering Design Process

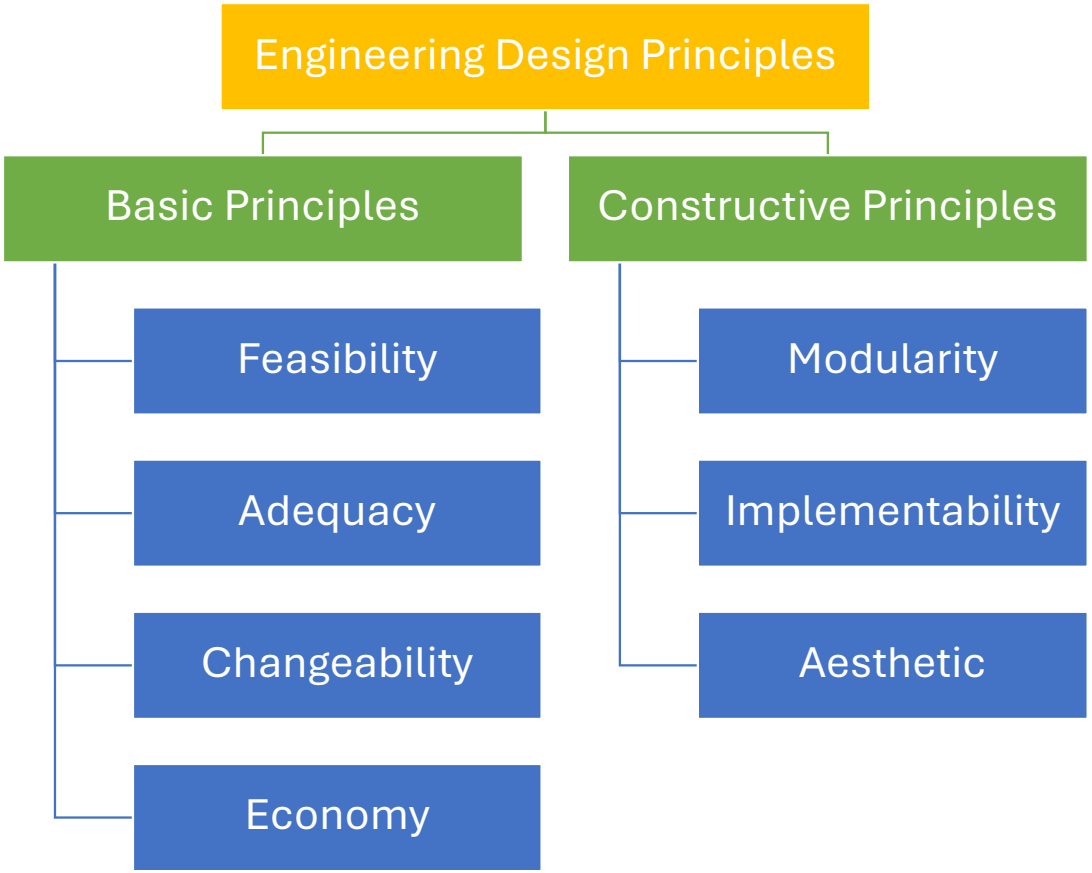


- Starts with SRS
- Generates many candidate architectures
- Evaluates candidates
- Leads to the creation of design document

Lecture slides are adapted from Christopher Fox, Introduction to Software Engineering Design – Processes, Principles, and Patterns with UML2, Pearson – Addison Wesley.

Software Engineering Design Principles

- **Design principles** are statements about what makes a design better.



Lecture slides are adapted from Christopher Fox, Introduction to Software Engineering Design – Processes, Principles, and Patterns with UML2, Pearson – Addison Wesley.

Software Engineering Design Principles (Cont.)

- **Basic principles** state characteristics that make a design better able to meet stakeholder needs and desires.
  - *Feasibility* - A design is acceptable only if it can be realized.
  - *Adequacy* - Designs that meet more stakeholder needs and desires, subject to constraints, are better.
  - *Economy* - Design that can be built for less money, in less time, with less risk, are better.
  - *Changeability* - Design that make a program easier to change are better.

Lecture slides are adapted from Christopher Fox, Introduction to Software Engineering Design – Processes, Principles, and Patterns with UML2, Pearson – Addison Wesley.

Software Engineering Design Principles (Cont.)

- **Constructive principles** state, based on experience, that certain engineering design characteristics make a design better.
  - *Modularity principles* - Good design are modular; these principles help evaluate whether designs specify good modules.
  - *Implementability principles* – Good designs are easier to build; these principles help evaluate whether designs will be easy to implement.
  - *Aesthetic principles* - Good design are beautiful; these principles help pick out beautiful designs.

Lecture slides are adapted from Christopher Fox, Introduction to Software Engineering Design – Processes, Principles, and Patterns with UML2, Pearson – Addison Wesley.

Modularity

- A *modular* program is composed of well-defined, conceptually simple, and independent units that communicate through well-defined interfaces.

Program
- Sub-programs or sub-systems
  - Packages, compilation units
    - Classes, functions
      - Attributes, operations, blocks
      - Lines of code

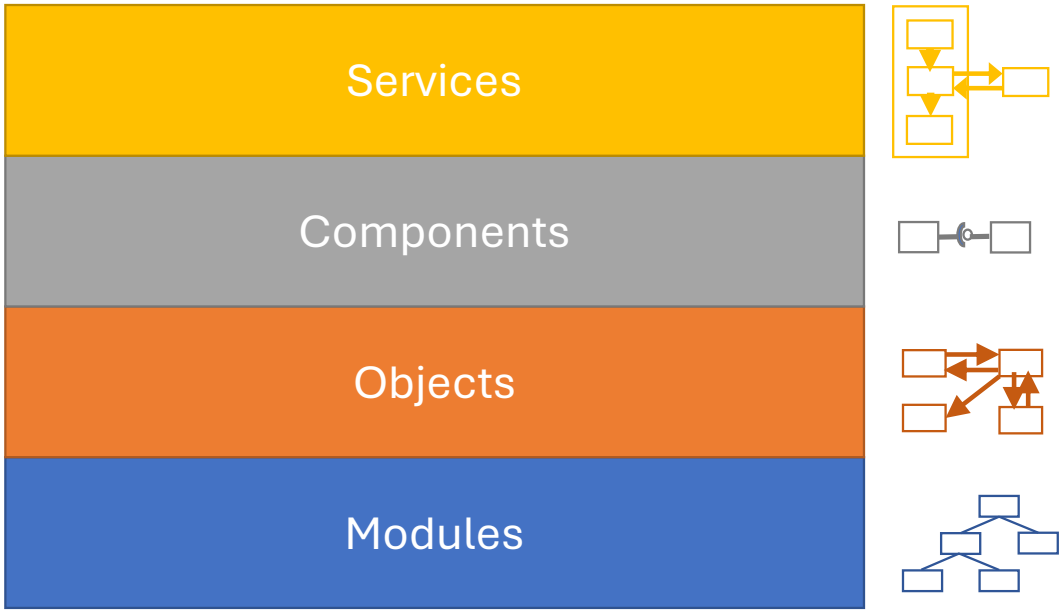
Lecture slides are adapted from Christopher Fox, Introduction to Software Engineering Design – Processes, Principles, and Patterns with UML2, Pearson – Addison Wesley.

Advantages of Modularity

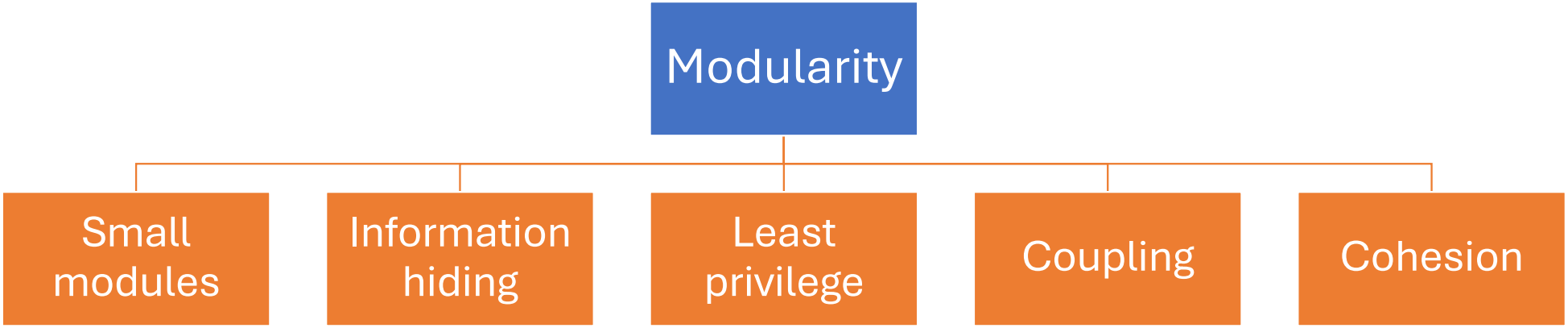
- Easier to understand and explain
- Easier to document
- Easier to change
- Easier to test and debug
- Easier to tune

Lecture slides are adapted from Christopher Fox, Introduction to Software Engineering Design – Processes, Principles, and Patterns with UML2, Pearson – Addison Wesley.

Software Modularity



Modularity Principles



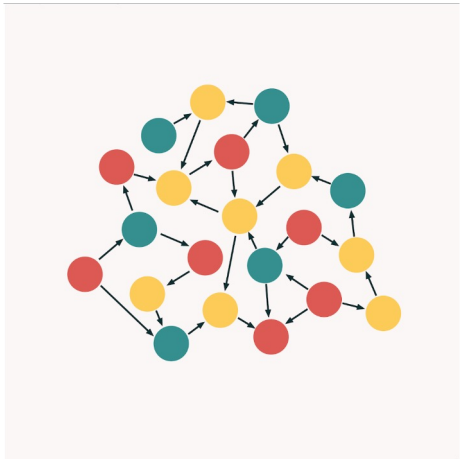
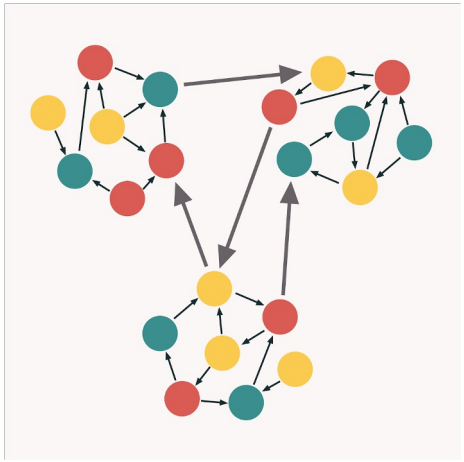
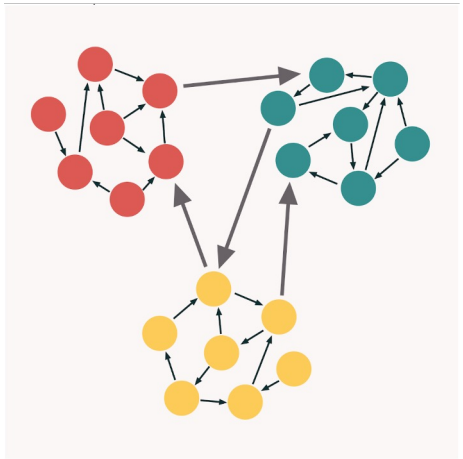
Modularity Principles (Cont.)

- ***Small modules***: Designs with small modules are better.
- ***Information Hiding***: Each module should shield the details of its internal structure and processing from other modules.
- ***Least Privilege***: Modules should not have access to unneeded resources.

Lecture slides are adapted from Christopher Fox, Introduction to Software Engineering Design – Processes, Principles, and Patterns with UML2, Pearson – Addison Wesley.

Modularity Principles (Cont.)

- ***Cohesion***: the "*degree of relatedness*" **within** that unit, module, object, or component.
- ***Coupling***: the "*degree of dependence*" **between** software units, modules or components.

Cohesion

Staff
checkEmail() sendEmail() emailValidate() printLetter()

VS

Staff
-salary -email
setSalary(salary) getSalary() setEmail(email) getEmail()



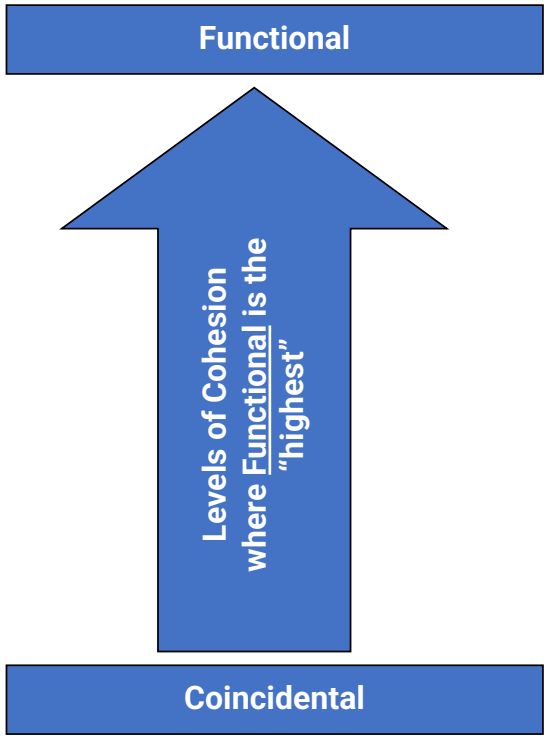
Low cohesion



High cohesion

https://stackoverflow.com/questions/3085285/difference-between-cohesion-and-coupling

Cohesion Level

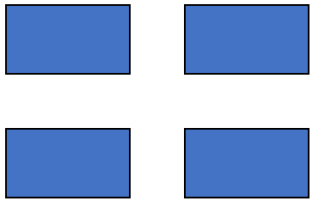


Performing 1 single function

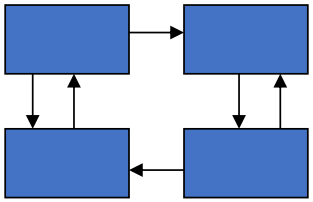
The higher the better

Performing more than 1 unrelated functions

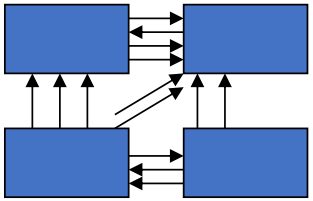
Coupling



No dependencies



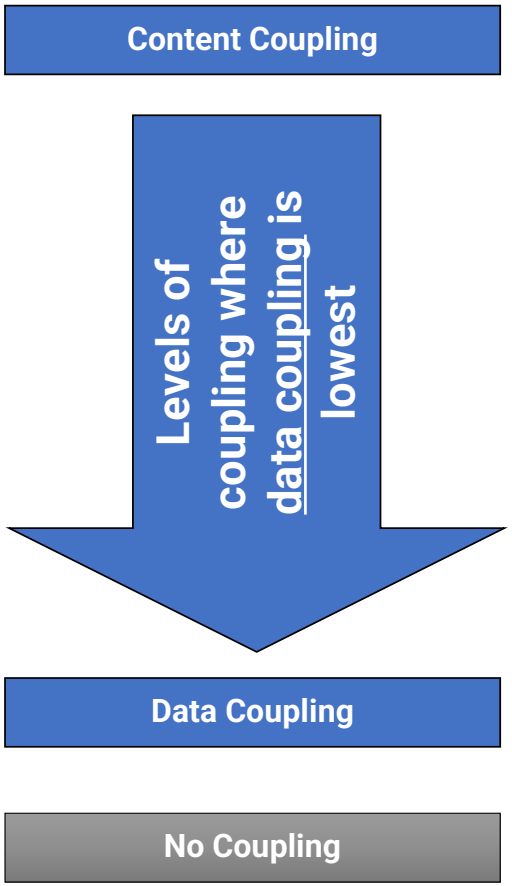
Loosely coupled-some dependencies



Highly coupled-many dependencies

High coupling makes modifying parts of the system difficult, e.g., modifying a component affects all the components to which the component is connected.

Coupling Level



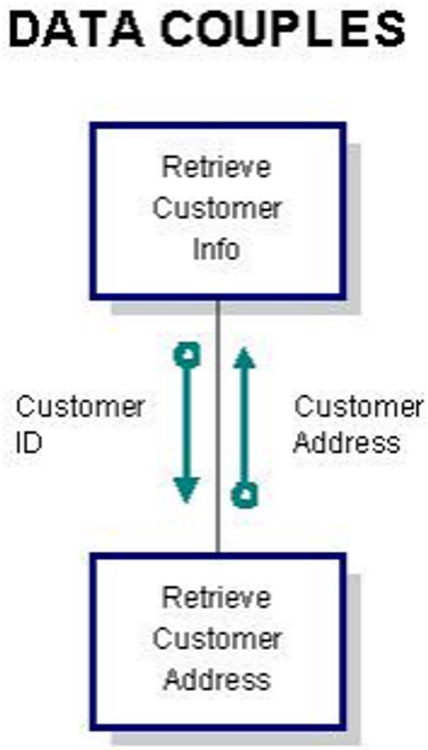
Accessing the internal data or procedural information

The lower the better

Passing only the necessary information
Ideal, but not practical

Data Coupling

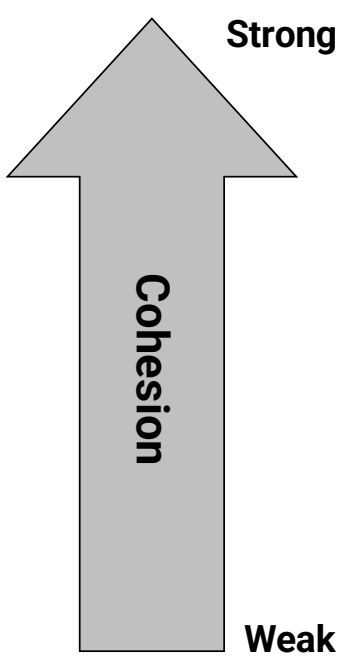
- Definition: Two components are data coupled if there are homogeneous data items
- Every argument is simple argument or data structure in which all elements are used
- Good, if it can be achieved
- Easy to write contracts for this and modify component independently

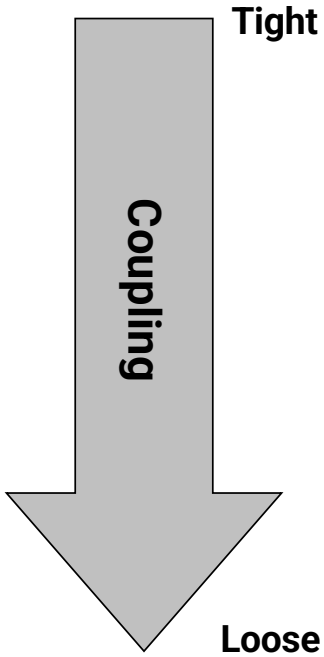


Reference: https://it.toolbox.com/blogs/craigborysowich/design-principles-coupling-data-and-otherwise-050307

Cohesion and Coupling

High Level





Low Level

Exercise

1. โมดูลหนึ่งทำหลายหน้าที่ เช่น อ่านไฟล์ แสดงผล และคำนวณข้อมูล

   -> _____

2. โมดูลหนึ่งเรียกใช้ตัวแปรภายในของอีกโมดูลโดยตรง

   -> _____

3. แต่ละโมดูลมีหน้าที่เฉพาะและไม่ซ้ำซ้อนกัน

   -> _____

4. โมดูลต่าง ๆ ติดต่อกันผ่าน interface ที่ชัดเจน

   -> _____

5. การเปลี่ยนโค้ดในโมดูลหนึ่งมักทำให้ต้องแก้โมดูลอื่นด้วย

   -> _____

6. โค้ดของโมดูลมีความเป็นเอกภาพ เน้นทำงานเดียวให้สมบูรณ์

   -> _____

Exercise

ข้อใดมีการออกแบบโมดูลได้ดีกว่า และดีกว่าในด้านใด

Design A

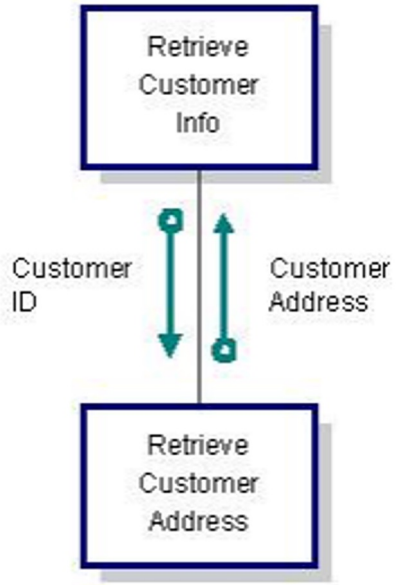
The print routine of the customer billing takes the customer's name, address, and billing information as an argument, parses it, and prints the name, address, and billing information.

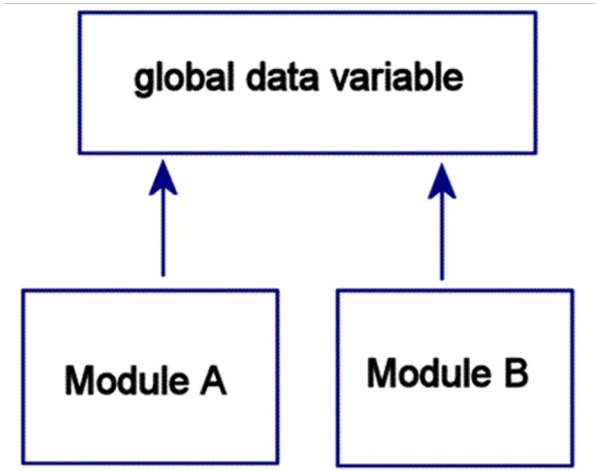
Design B

The print routine of the customer billing accepts a customer data structure as an argument, parses it, and prints the name, address, and billing information.

Exercise

ข้อใดมีการออกแบบโมดูลได้ดีกว่า และดีกว่าในด้านใด





Exercise

ข้อใดมีการออกแบบโมดูลได้ดีกว่า และดีกว่าในด้านใด

Design A

The FileReader module is designed to handle file input operations. It includes methods for opening, reading, and closing files.

Design B

The DataProcessor module combines various functionalities within a single unit. It includes methods for reading data from files, performing mathematical calculations, and sending email notifications.

Exercise

ข้อใดมีการออกแบบโมดูลได้ดีกว่า และดีกว่าในด้านใด

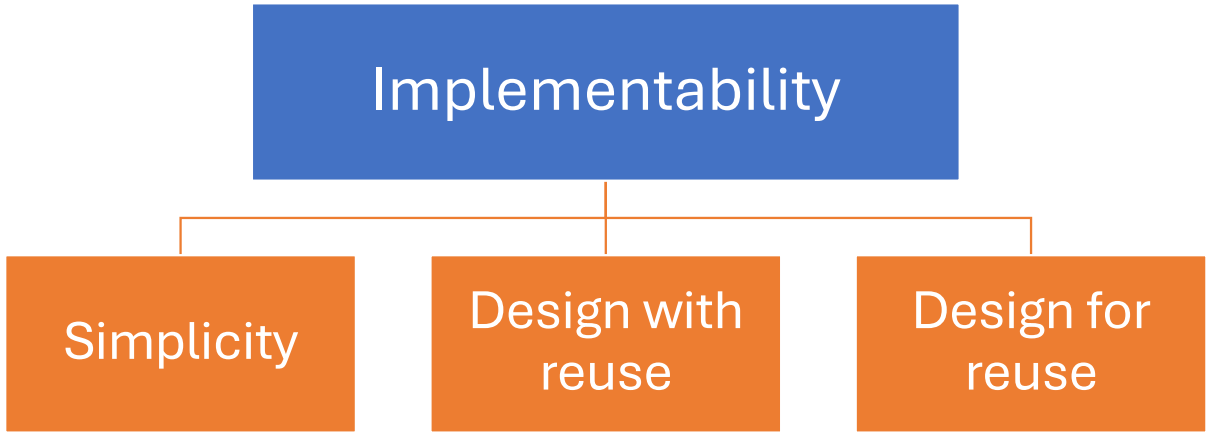
Design A

The Utility module is a catch all for various functionalities, ranging from file operations and mathematical calculations to user interface interactions and data processing. It attempts to accommodate a wide range of unrelated tasks within a single unit.

Design B

The EmployeeDatabase module serves as the core component for managing employee information within our HR system. It encapsulates functionalities related to employee data, including adding new employees, updating existing records, generating reports, and calculating payroll. All methods and functions within this module are tightly focused on handling employee-related tasks.

Implementability Principles



- ***Simplicity***: Simpler designs are better.
- ***Design with reuse***: Designs that reuse existing artifacts are better.
- ***Design for reuse***: Designs that produce reusable artifacts are better.

Lecture slides are adapted from Christopher Fox, Introduction to Software Engineering Design – Processes, Principles, and Patterns with UML2, Pearson – Addison Wesley.

Aesthetic Principles

- ***Beauty***: This refers to the designs that are simple, clear and free of unnecessary complexity. It avoids redundancy, awkward structures, and overly complicated logic. Not about the user interface.
- ***Symmetry***: Designs that are consistent across the system. Similar components should be designed in similar ways.
- ***Balance***: Refers to even distribution of responsibilities and complexity across components. Avoid "God classes" (one class does everything).
- ***Proportion***: Refers to an appropriate sizing and scaling of components relative to their purpose. Each module should be neither too large (complex) nor too small (fragmented).

Lecture slides are adapted from Christopher Fox, Introduction to Software Engineering Design – Processes, Principles, and Patterns with UML2, Pearson – Addison Wesley.

Example (Beauty principle)

Java

```java
public int calc(int a, int b, String op) {
    if (op.equals("add")) {
        return a + b;
    } else if (op.equals("sub")) {
        return a - b;
    } else if (op.equals("mul")) {
        return a * b;
    } else {
        return 0;
    }
}
```

- Hardcoded logic
- Not extensible
- Low readability

Java

```java
interface Operation {
    int execute(int a, int b);
}

class Add implements Operation {
    public int execute(int a, int b) { return a + b; }
}

class Multiply implements Operation {
    public int execute(int a, int b) { return a * b; }
}
```

- Simple
- Extensible
- Clear separation of concerns

Relationship Among Principles

- Some principles usually support one another.
  - Cohesion, Coupling, Information Hiding
  - Simplicity, Beauty
  - Simplicity, Small Modules
- Some principles are often in conflict.
  - Simplicity, Design for Reuse
  - Small Modules, Design for Reuse
  - Cohesion, Small Modules
- Support or conflict may depend on the case at hand.

Lecture slides are adapted from Christopher Fox, Introduction to Software Engineering Design – Processes, Principles, and Patterns with UML2, Pearson – Addison Wesley.

Summary

- The intent of software design is to apply a set of principles, concepts, and practices that lead to the development of a high-quality system or product. The goal is to create a model of software that will implement all customer requirements.
- Software architecture provides a holistic view of the system, depicting structure and organization of software components, their properties, and the connections between them.
- Engineering design principles guides how to design the software components.